Infraestrutura PostgreSQL configurada em Multi-AZ para garantir a resiliência do sistema.Hardware: 4 vCPUs, 32 GB de memória e SSD (preferencialmente NVME) com IOPS garantidos. Gerenciamento de acessos PgBouncer configurado em modo transação para evitar o consumo de conexões do banco de dados na proporção que cresce a demanda pelo software. Arquitetura focada em performance.

Operações idempotentes por meio de INSERT ... ON CONFLICT para tratar erros de rede e retransmissões. Gerenciamento de sobrecarga (fluxos assíncronos) Filas (Redis Streams, SQS ou RabbitMQ) entre o banco e uma API para dados menos críticos, como notificações, logs e históricos. Trabalhadores consomem essas filas e fazem gravação em bloco. Dados necessários, como orçamentos e agendamentos, devem ser processados sincronizadamente. Particionamento de dados Uso de PARTITION BY RANGE (via pg_partman) baseado em dados para tabelas de grande volume, como logs, histórico de Agendamento e Alerta_animais. Remoção ou arquivo de partições antigas para melhorar o desempenho do VACUUM. Otimização do motor PostgreSQL Definição de shared_buffers em cerca de 25% da RAM, aumento do max_wal_size e configuração de checkpoint_timeout entre 10 e 15 minutos. Ativação de wal_compression. Configuração de Autovacuum intensivo para tabelas de alta atividade. Desativação do synchronous_commit apenas para registros de log, onde uma perda de milissegundos é aceitável. Separação de leitura e escrita Uso de réplicas para o Mapa Pet, relatórios e consultas. Cache de buscas recorrentes no Redis. PostGIS com índices GiST para as funções geográficas do Mapa Pet. Durabilidade e desastre Backups automáticos com PITR (via arquivo de WAL). Métricas de recuperação: RTO máximo de 30 minutos e RPO de até 5 minutos. Testes e monitoramento Testes de carga com pgbench, JMeter ou k6 para verificar os limites definidos. Monitoramento via pg_stat_statements com monitoramento de travamentos, atrasos de replicação, IOPS e CPU. Infraestrutura de acesso.

O uso de chaves específicas e comandos INSERT ... ON CONFLICT garantem que requisições repetidas não criem duplicidade de dados. Controle de vazão (limitação

de taxa). A API usa um gateway para limitar o número de chamadas que um usuário pode fazer, protegendo o banco de dados de sobrecargas pontuais. Expansão lateral. A API é stateless e usa um balanceador de carga, permitindo que o número de servidores cresça com escalonamento automático conforme necessário. Um PostgreSQL gerenciado com Multi-AZ, PgBouncer, gravação em lote, fila para dados não críticos, particionamento por dados e réplicas de leitura suporta 1 milhão de escritas sem sharding.